



SUSTAINABLE WASTE MANAGEMENT ASSISTANT USING GENERATIVE AI

**(EcoSense – AI-Powered Waste Classification, Recycling Guidance &
Collection Center Locator)**

Project Documentation

Submitted By

Vipparla Rakesh

Roll Number: 23P31A4271

Department of Artificial Intelligence and Machine Learning

Aditya College of Engineering and Technology

Smart City | Generative AI | Full Stack Web Application

React.js • Python • Flask • Firebase Firestore

Groq AI • Leaflet.js • OpenStreetMap • Chart.js

Internship Partner

SmartBridge SkillWallet Academy

Academic Year

2026–2027

Team Details

S.No	Name	Roll Number	Role	Email
1	Vipparla Rakesh	23P31A4271	Team Lead	rkraakrakesh1000@gmail.com
2	Kodamanchili Sri Satya Lokesh	23P31A4237	Member	satyalokesh970@gmail.com

Project Description

EcoSense – Sustainable Waste Management Assistant Using Generative AI is an AI-powered smart city waste management platform designed to help citizens identify household waste, receive proper disposal instructions, locate nearby recycling centers, and monitor recycling activities through an intelligent dashboard. The application promotes environmental sustainability by encouraging responsible waste segregation and recycling practices using Artificial Intelligence.

The platform enables users to classify household waste items, receive AI-generated recycling guidance, discover nearby waste collection centers, monitor waste disposal history, and analyze recycling performance through interactive analytics. By integrating modern web technologies, cloud services, and Generative AI, EcoSense provides an intelligent and user-friendly solution for sustainable urban waste management.

The application is developed using **React.js** for the frontend, **Python Flask** for backend processing, and **Firebase Firestore** as the cloud database. It integrates the **Groq API** with the **LLaMA 3.3-70B Versatile** model to classify waste items, generate disposal instructions, provide recycling recommendations, and identify hazardous materials. **Leaflet.js** with OpenStreetMap is used to display nearby waste collection centers, while **Chart.js** provides real-time analytics and dashboard visualizations. The frontend is deployed using **GitHub Pages**, and the backend is deployed on **Render**.

EcoSense provides a responsive, secure, and easy-to-use interface that enables users to scan household waste, understand recycling procedures, locate nearby disposal facilities, maintain scan history, and monitor recycling statistics. The project demonstrates the practical application of **Artificial Intelligence**, **Cloud Computing**, **Geographic Information Systems (GIS)**, and **Full Stack Web Development** to address real-world environmental challenges and promote smart city sustainability.

Key Features

- AI-Powered Waste Classification
- Smart Disposal Guidance
- Recycling Instructions Generation
- Hazardous Waste Detection
- Interactive Collection Center Map
- Personal Scan History
- Dashboard Analytics
- Firebase Cloud Database
- Responsive Web Interface

Scenarios

Scenario 1: AI-Powered Waste Classification

A resident finds an old battery at home and is unsure how to dispose of it safely. They open the EcoSense application and enter "**battery**" into the AI Waste Scanner. The system sends the request to the Groq AI engine, which classifies the item as **Hazardous Waste** and generates detailed disposal instructions, hazard warnings, recycling recommendations, and environmental safety tips. The result is automatically stored in the user's scan history for future reference.

Scenario 2: Smart Recycling Guidance

A user wants to recycle a **plastic bottle** responsibly. They search for the item using the EcoSense Waste Scanner. The AI identifies it as **Plastic Waste**, confirms that it is recyclable, and provides step-by-step recycling instructions, including rinsing the bottle, removing the cap, and placing it in the appropriate recycling bin. The application also suggests eco-friendly alternatives, such as using reusable bottles to reduce plastic waste.

Scenario 3: Nearby Waste Collection Centers

After scanning a waste item, the user navigates to the **Collection Map** section. EcoSense displays nearby recycling centers and hazardous waste collection facilities using **Leaflet.js** and **OpenStreetMap**. Users can view each center's location, accepted waste types, operating hours, and directions, making it easier to dispose of waste responsibly and safely.

Scenario 4: Waste Analytics Dashboard

The user opens the **Analytics Dashboard** to review their waste management activities. EcoSense displays interactive charts showing the total number of waste scans, recyclable items, hazardous waste detected, recycling rate, and category-wise waste statistics. These insights help users monitor their environmental impact, improve recycling habits, and encourage sustainable waste management practices.

Technical Architecture

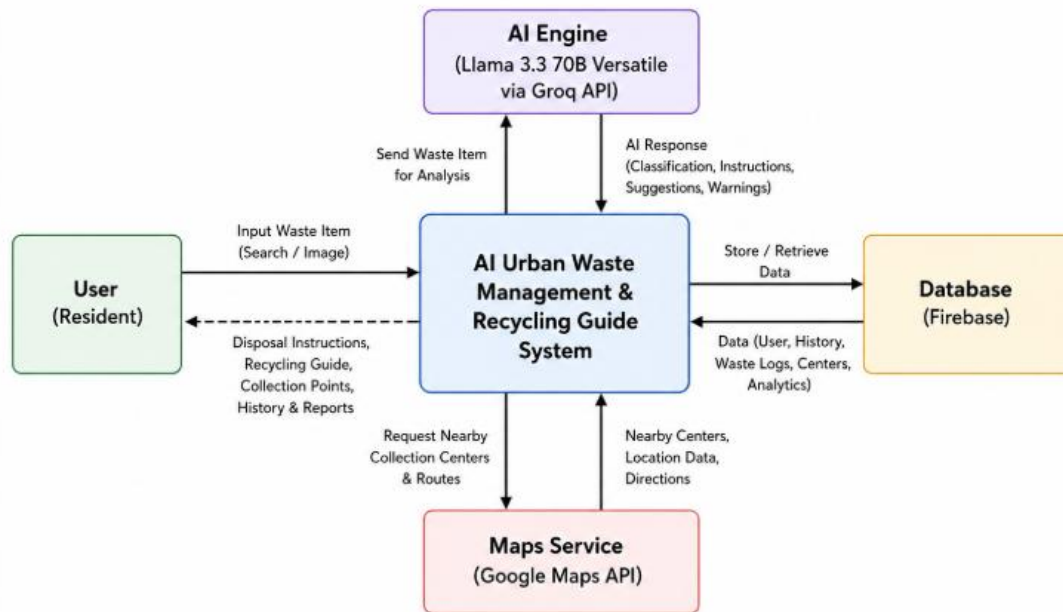


Figure 4.1: Technical Architecture of EcoSense – Sustainable Waste Management Assistant Using Generative AI

Technical Architecture

EcoSense follows a modern multi-tier architecture that integrates a responsive web-based frontend, a Python Flask backend, cloud-based data storage, Artificial Intelligence services, and location-based mapping technologies. The architecture is designed to provide secure, scalable, and efficient waste management while ensuring seamless communication between users, cloud services, and AI-powered processing modules.

The frontend is developed using **React.js**, HTML, CSS, and JavaScript, providing an intuitive and responsive interface for waste scanning, recycling guidance, dashboard analytics, collection center mapping, and history management. Users interact with the application through this interface to classify waste items, receive disposal instructions, locate nearby recycling facilities, and monitor recycling activities.

The application communicates with a **Python Flask Backend**, which acts as the core processing service. The backend validates user requests, manages application logic, processes AI prompts, communicates with external services, and performs read/write operations with the cloud database.

Waste scan history, recycling records, analytics data, and user information are securely stored in **Firebase Firestore**, enabling centralized cloud storage, real-time synchronization, and efficient retrieval of waste management records.

EcoSense integrates the **Groq API** using the **LLaMA 3.3-70B Versatile** large language model to classify waste items, generate recycling instructions, provide disposal guidance, identify hazardous waste, and recommend eco-friendly alternatives. Prompt engineering techniques are used to transform user input into accurate and easy-to-understand waste management guidance.

The system also integrates **Leaflet.js** with **OpenStreetMap** to display nearby recycling centers and hazardous waste collection facilities. **Chart.js** is used to generate real-time analytics, helping users monitor waste classification trends, recycling statistics, and environmental impact.

The frontend is deployed using **GitHub Pages**, while the backend services are deployed on **Render**, enabling secure communication between all application components through RESTful APIs over the internet.

Architecture Components

Frontend

- React.js
- HTML5
- CSS3
- JavaScript
- Responsive User Interface

Backend

- Python Flask
- REST API
- Business Logic
- Input Validation

Cloud Database

- Firebase Firestore
- Waste Scan History
- User Records
- Analytics Data

Artificial Intelligence

- Groq API
- LLaMA 3.3-70B Versatile
- AI Waste Classification
- Disposal Guidance Generation

External Services

- Leaflet.js

- OpenStreetMap
 - Chart.js
 - GitHub Pages
 - Render
-

Pre-requisites

Introduction

Before developing and deploying the **EcoSense – Sustainable Waste Management Assistant Using Generative AI**, developers should have basic knowledge of web development, cloud services, Artificial Intelligence, and API integration. The following software, tools, frameworks, and cloud platforms are required to successfully build, test, and deploy the application.

Programming Knowledge

- HTML5
 - CSS3
 - JavaScript (ES6)
 - React.js
 - Python
 - Flask Framework
 - REST APIs
 - Prompt Engineering
 - Generative Artificial Intelligence (GenAI)
-

Development Tools

- Visual Studio Code
 - Git
 - GitHub
 - Google Chrome / Microsoft Edge
 - Postman (API Testing)
 - Firebase Console
-

Cloud Services

- Firebase Firestore
- Firebase Authentication

- Render Cloud Platform
 - GitHub Pages
 - Groq Cloud API
 - OpenStreetMap
 - Leaflet.js
-

AI Requirements

- Groq API Key
 - LLaMA 3.3-70B Versatile Model
 - Prompt Engineering Knowledge
 - Secure API Key Configuration
-

Hardware Requirements

- Processor: Intel Core i5 (10th Generation or above) / AMD Ryzen 5 or equivalent
 - RAM: Minimum 8 GB (16 GB Recommended)
 - Storage: Minimum 256 GB SSD
 - Stable Internet Connection (10 Mbps or above)
-

Software Requirements

- Windows 10 / Windows 11
- macOS Monterey or later
- Ubuntu 20.04 or later
- Google Chrome
- Mozilla Firefox
- Microsoft Edge

Project Workflow

The development of **EcoSense – Sustainable Waste Management Assistant Using Generative AI** follows a structured software development workflow that divides the project into multiple milestones. Each milestone focuses on a specific phase of the software development lifecycle, including requirement analysis, system architecture, feature implementation, backend and frontend development, deployment, testing, and project completion. This organized approach ensures efficient development, better project management, and successful implementation of intelligent waste management functionalities.

Milestone 1: Requirement Analysis and Architecture

- **Activity 1.1:** Generate and Configure the Groq AI API Key.
 - **Activity 1.2:** Configure Required Accounts and Cloud Services.
 - **Activity 1.3:** Research and Select the Appropriate Generative AI Model.
 - **Activity 1.4:** Define the Architecture of the Application.
 - **Activity 1.5:** Set Up the Development Environment.
-

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop Core Waste Management Features.
 - **Activity 2.2:** Implement the Python Flask Backend.
-

Milestone 3: Backend and Application Development

- **Activity 3.1:** Writing the Main Application Logic.
 - **Activity 3.2:** Integrate AI Waste Classification Services.
-

Milestone 4: Frontend Development

- **Activity 4.1:** Designing and Developing the User Interface.
 - **Activity 4.2:** Creating Dynamic Content Rendering with React.js.
-

Milestone 5: Integration and Deployment

- **Activity 5.1:** Preparing the Application for Local Deployment.
 - **Activity 5.2:** Deploy the Frontend and Backend Services.
 - **Activity 5.3:** Validate Deployment and Test Live Workflow.
-

Milestone 6: Conclusion

- **Activity 6.1:** Summarize the Overall Project Objectives and Achievements.

- **Activity 6.2:** Evaluate the Performance and Reliability of the System.
- **Activity 6.3:** Highlight the Benefits of AI-Based Waste Management.
- **Activity 6.4:** Identify Project Limitations and Future Enhancements.
- **Activity 6.5:** Complete Final Documentation, Testing, and Project Review.

Milestone 1: Requirement Analysis and Architecture

The Requirement Analysis and Architecture phase forms the foundation of the **EcoSense – Sustainable Waste Management Assistant Using Generative AI** project. During this phase, the project requirements were carefully analyzed, cloud services were configured, and the overall system architecture was designed. The development environment was prepared to ensure smooth communication between the frontend, backend, cloud database, Artificial Intelligence services, and mapping technologies. Proper planning during this milestone established a scalable, secure, and efficient smart waste management platform.

Activity 1.1: Generate and Configure the AI Service API Key

To enable Artificial Intelligence capabilities within EcoSense, a **Groq API key** was generated through the Groq Developer Console. The API key was securely configured and integrated into the application to enable communication with the Groq AI service. The application uses the **LLaMA 3.3-70B Versatile** model to classify waste items, generate recycling instructions, identify hazardous waste, and provide eco-friendly disposal recommendations. The API credentials were securely stored to prevent unauthorized access while ensuring reliable AI-powered processing.

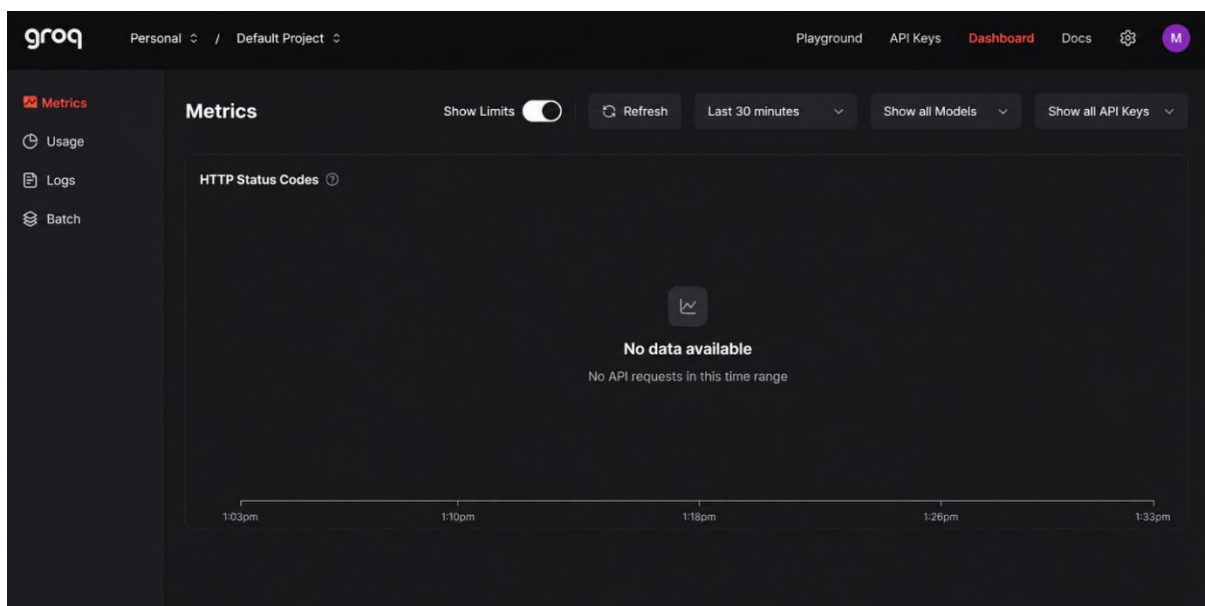
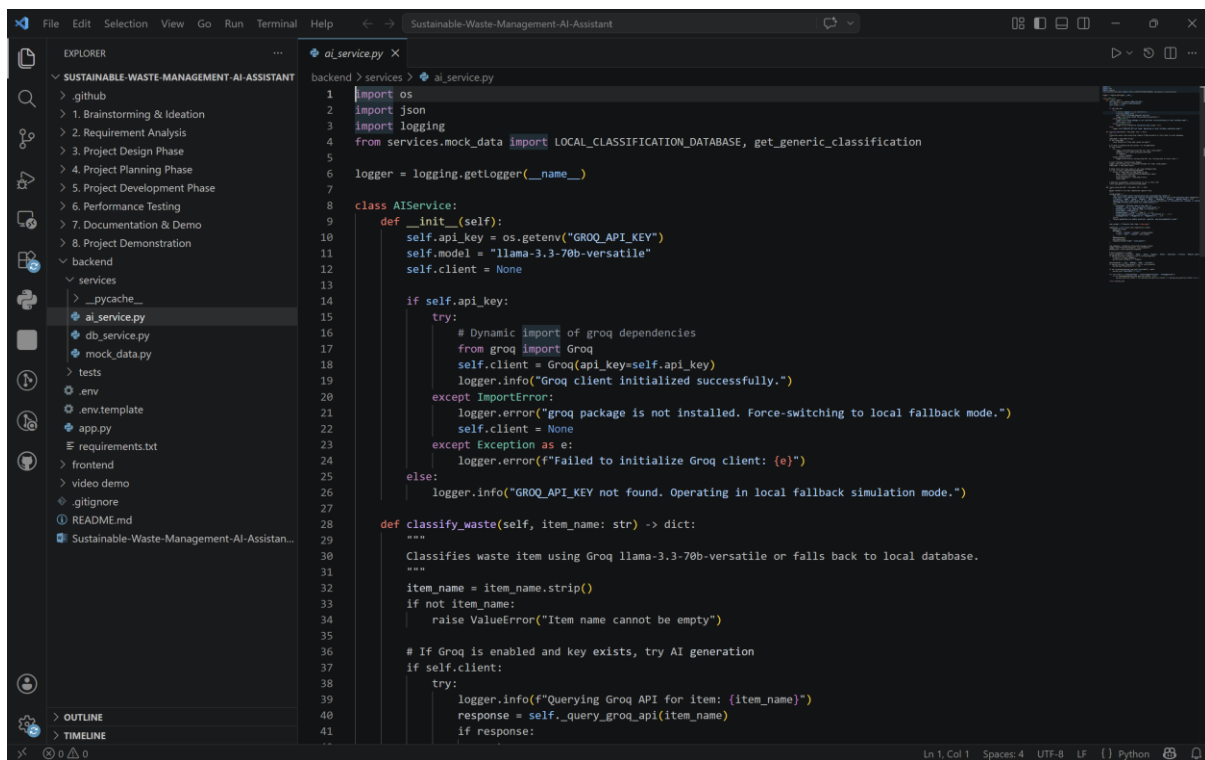


Figure 7.1: Groq Dashboard for API Key Configuration

Activity 1.2: Configure Required Accounts and Services

Before development, the required tools and services were configured to support application development and deployment. Visual Studio Code was used as the primary development environment, GitHub was configured for source code management and version control, and the Groq API was integrated to provide AI-powered waste classification and disposal guidance.

The project also supports Firebase Firestore for cloud storage; however, it includes an offline JSON fallback mechanism that allows the application to function even when Firebase credentials are not configured. This approach improves reliability and simplifies local development and testing.



```

1 import os
2 import json
3 import logging
4 from services.mock_data import LOCAL_CLASSIFICATION_DATABASE, get_generic_classification
5
6 logger = logging.getLogger(__name__)
7
8 class AIService:
9     def __init__(self):
10         self.api_key = os.getenv("GROQ_API_KEY")
11         self.model = "llama-3.3-70b-versatile"
12         self.client = None
13
14     if self.api_key:
15         try:
16             # Dynamic import of groq dependencies
17             from groq import Groq
18             self.client = Groq(api_key=self.api_key)
19             logger.info("Groq client initialized successfully.")
20         except ImportError:
21             logger.error("groq package is not installed. Force-switching to local fallback mode.")
22             self.client = None
23         except Exception as e:
24             logger.error(f"Failed to initialize Groq client: {e}")
25     else:
26         logger.info("GROQ_API_KEY not found. Operating in local fallback simulation mode.")
27
28     def classify_waste(self, item_name: str) -> dict:
29         """
30         Classifies waste item using Groq llama-3.3-70b-versatile or falls back to local database.
31         """
32         item_name = item_name.strip()
33         if not item_name:
34             raise ValueError("Item name cannot be empty")
35
36         # If Groq is enabled and key exists, try AI generation
37         if self.client:
38             try:
39                 logger.info(f"Querying Groq API for item: {item_name}")
40                 response = self._query_groq_api(item_name)
41                 if response:

```

Figure 7.2: Development Environment and Service Configuration

Activity 1.3: Research and Select the Appropriate Generative AI Model

Several Generative Artificial Intelligence models were evaluated based on response quality, inference speed, reasoning capability, API availability, and suitability for waste classification and recycling guidance. After comparing different Large Language Models, the **LLaMA 3.3-70B Versatile** model provided through the **Groq API** was selected due to its high inference speed, strong reasoning capabilities, and ability to generate accurate and user-friendly waste disposal recommendations. The selected model effectively classifies household waste, identifies hazardous materials, generates recycling instructions, and provides environmentally responsible disposal guidance.

Activity 1.4: Define the Architecture of the Application

The EcoSense system architecture was designed to establish secure communication between the React.js frontend, Python Flask backend, Artificial Intelligence engine, cloud database, and external mapping services. The frontend provides an intuitive interface for waste scanning, recycling guidance, analytics, and map visualization, while the Flask backend processes user requests, manages business logic, and communicates with external APIs. The Groq AI service performs intelligent waste classification and disposal guidance, while Firebase Firestore (or the offline JSON fallback mechanism) securely stores waste scan history and application data.

Leaflet.js integrated with OpenStreetMap enables users to locate nearby recycling and hazardous waste collection centers.

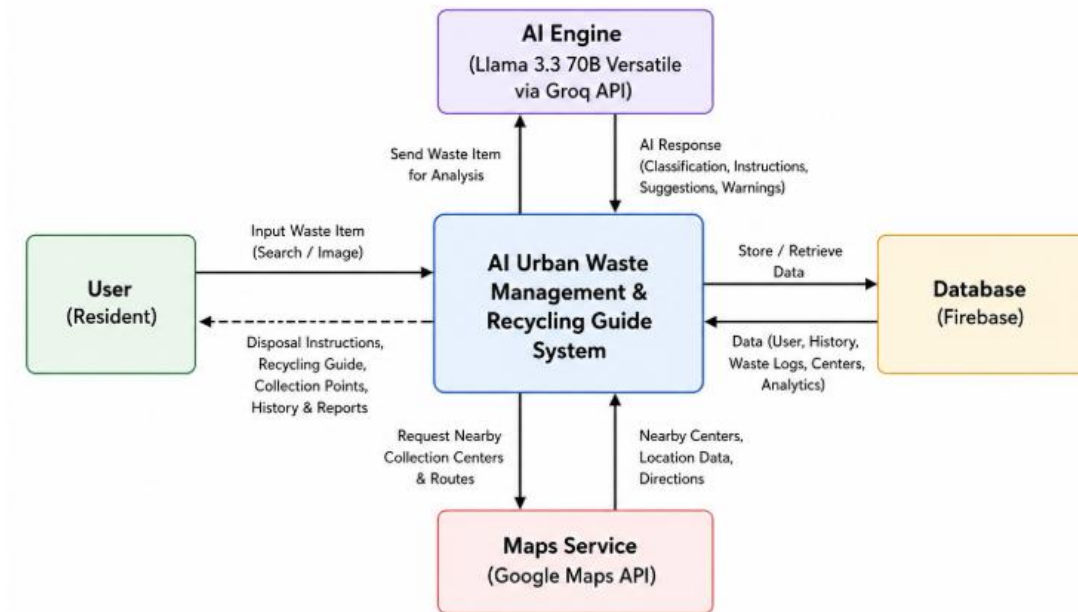
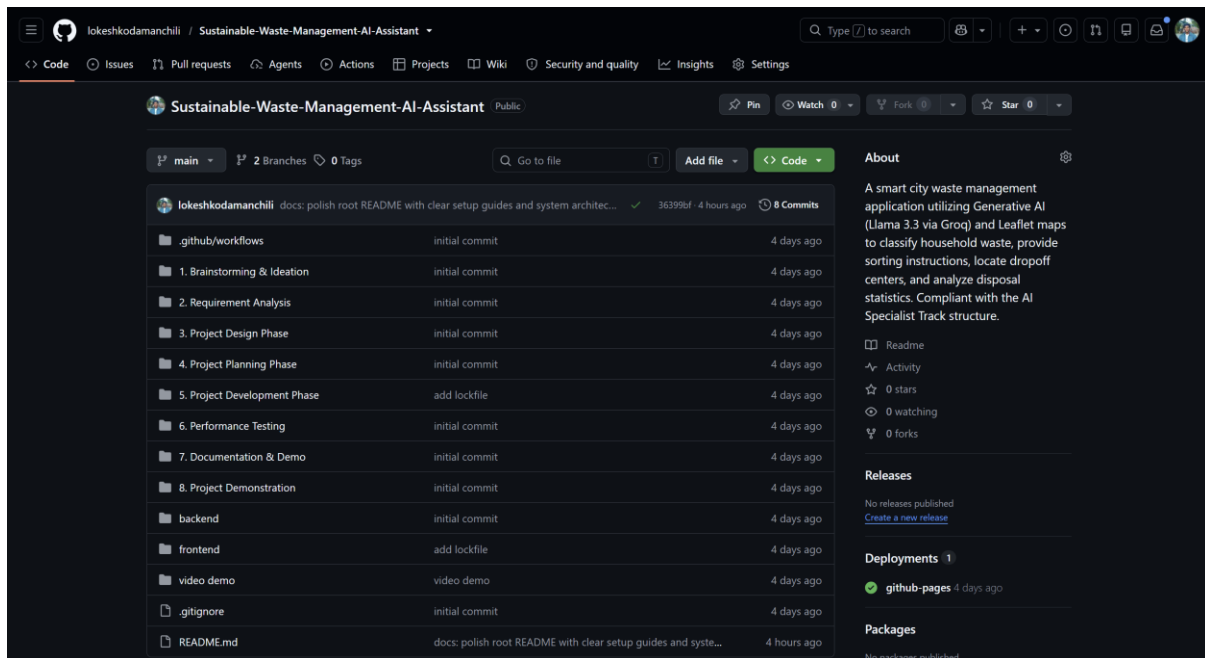


Figure 7.3: EcoSense Technical Architecture

Activity 1.5: Set Up the Development Environment

The complete development environment was prepared by installing **Visual Studio Code**, configuring the **GitHub repository**, setting up the **Python Flask backend**, developing the **React.js frontend**, integrating the **Groq API**, and configuring project dependencies. The application was tested locally using the Flask development server and React development server before deployment. The frontend was later deployed using **GitHub Pages**, while the backend services were deployed on **Render**, enabling efficient development, testing, debugging, and cloud deployment of the EcoSense platform.



Sustainable-Waste-Management-AI-Assistant Public

lokeshkodamanchili docs: polish root README with clear setup guides and system architec... 36399bf · 4 hours ago · 8 Commits

File	Commit	Time
.github/workflows	initial commit	4 days ago
1. Brainstorming & Ideation	initial commit	4 days ago
2. Requirement Analysis	initial commit	4 days ago
3. Project Design Phase	initial commit	4 days ago
4. Project Planning Phase	initial commit	4 days ago
5. Project Development Phase	add lockfile	4 days ago
6. Performance Testing	initial commit	4 days ago
7. Documentation & Demo	initial commit	4 days ago
8. Project Demonstration	initial commit	4 days ago
backend	initial commit	4 days ago
frontend	add lockfile	4 days ago
video demo	video demo	4 days ago
.gitignore	initial commit	4 days ago
README.md	docs: polish root README with clear setup guides and syste...	4 hours ago

About

A smart city waste management application utilizing Generative AI (Llama 3.3 via Groq) and Leaflet maps to classify household waste, provide sorting instructions, locate dropoff centers, and analyze disposal statistics. Compliant with the AI Specialist Track structure.

Releases

No releases published
[Create a new release](#)

Deployments 1

github-pages 4 days ago

Packages

No packages published

Figure 7.4: EcoSense GitHub Repository

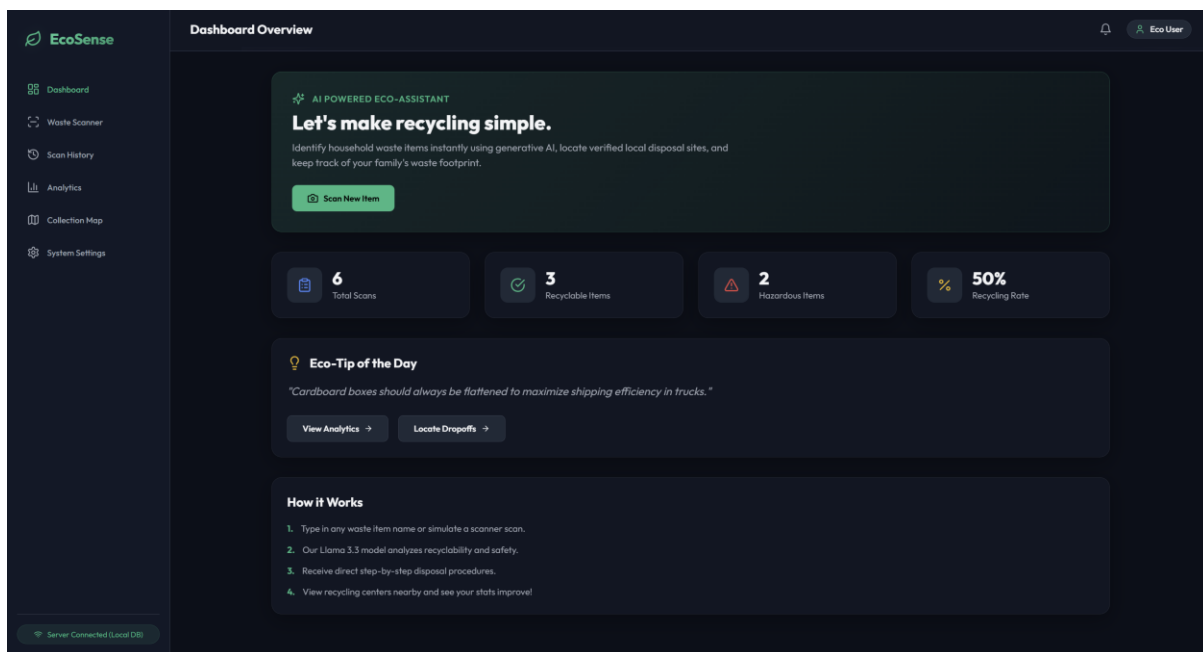
Milestone 2: Core Functionalities Development

The Core Functionalities Development phase focuses on implementing the essential modules of the EcoSense – Sustainable Waste Management Assistant Using Generative AI application. During this phase, the waste scanning system, AI-powered waste classification, recycling guidance, interactive collection center map, analytics dashboard, and scan history modules were developed. These features work together to provide users with a centralized platform for identifying waste items, learning proper disposal methods, and promoting environmentally responsible waste management.

Activity 2.1: Develop Core Waste Management Features

The core waste management modules were developed to simplify household waste identification, recycling guidance, disposal recommendations, and environmental awareness. The application provides a responsive dashboard where users can access all EcoSense services from a single interface. Each module has been designed with a clean and user-friendly layout to ensure smooth navigation across desktop and mobile devices.

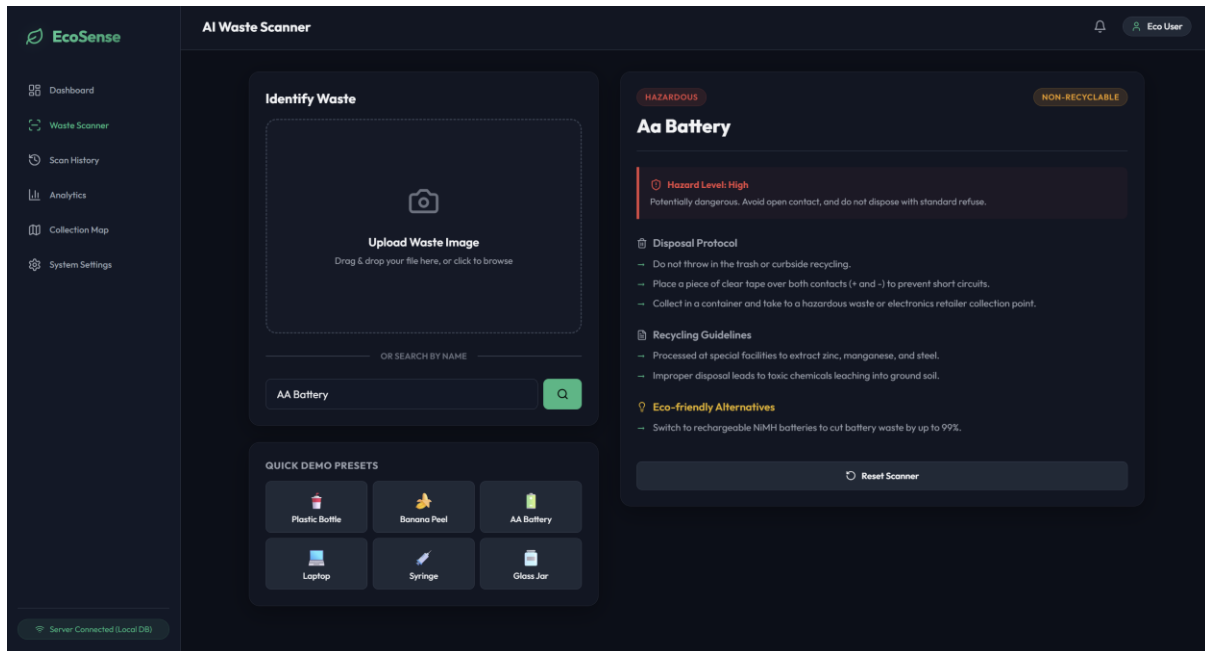
Figure 8.1: EcoSense Dashboard



AI Waste Scanner

The AI Waste Scanner allows users to enter the name of any household waste item and instantly receive an AI-generated classification. The Groq-powered LLaMA 3.3-70B Versatile model identifies the waste category, determines whether the item is recyclable or hazardous, and provides clear disposal instructions. This intelligent scanner helps users make environmentally responsible disposal decisions while reducing waste contamination.

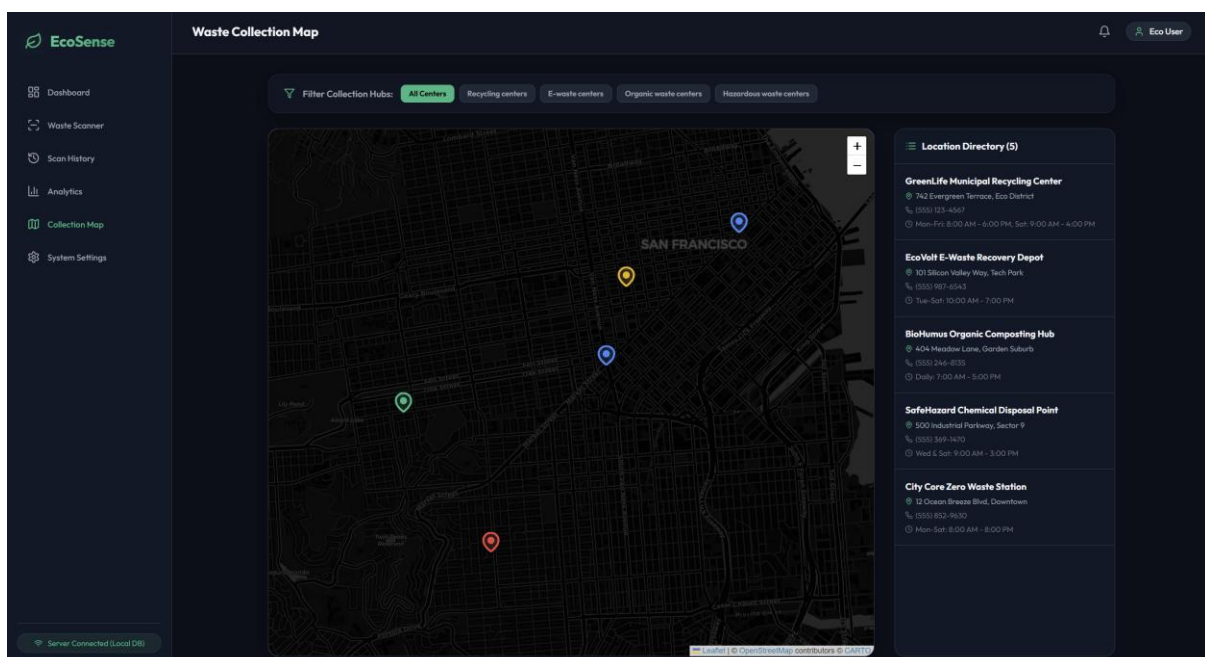
Figure 8.2: AI Waste Scanner Module



Collection Center Map

The Collection Center Map enables users to locate nearby recycling facilities, hazardous waste centers, composting locations, and e-waste collection points. Built using Leaflet.js with OpenStreetMap, the map displays color-coded markers and provides location details, helping users easily find appropriate waste disposal centers without requiring paid map services.

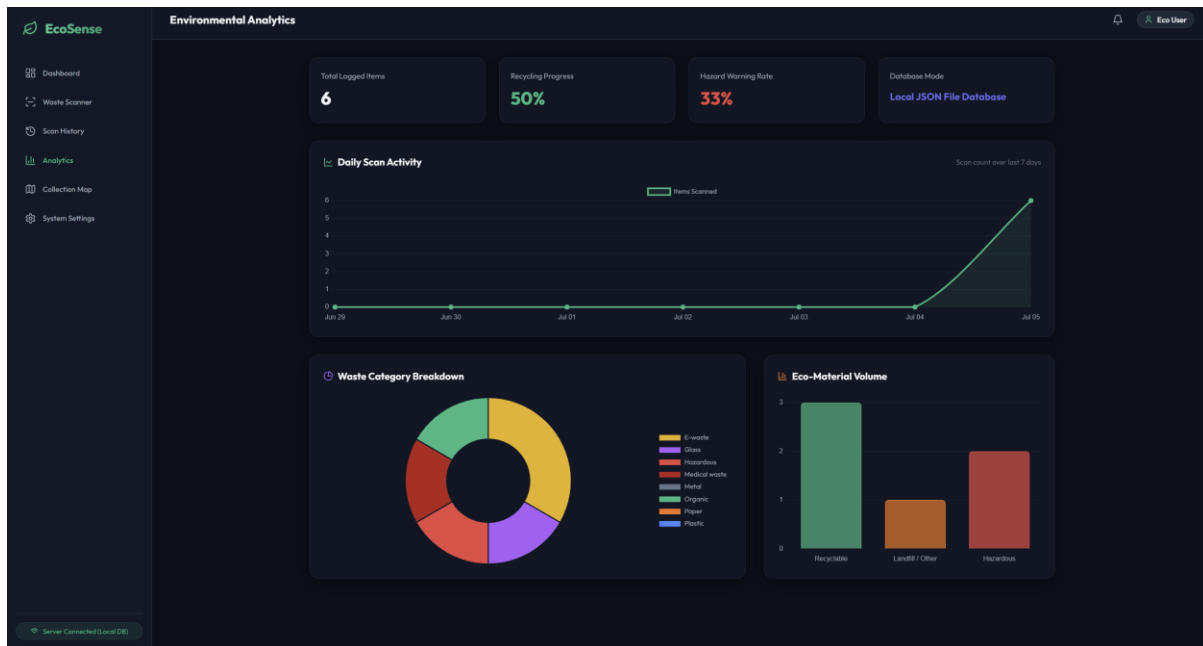
Figure 8.3: Collection Center Map



Analytics Dashboard

The Analytics Dashboard provides users with visual insights into their waste disposal activities. Interactive charts display recyclable and non-recyclable waste distribution, hazardous waste statistics, category-wise scan history, and weekly waste trends. These analytics encourage users to improve recycling habits and monitor their environmental impact over time.

Figure 8.4: Waste Analytics Dashboard



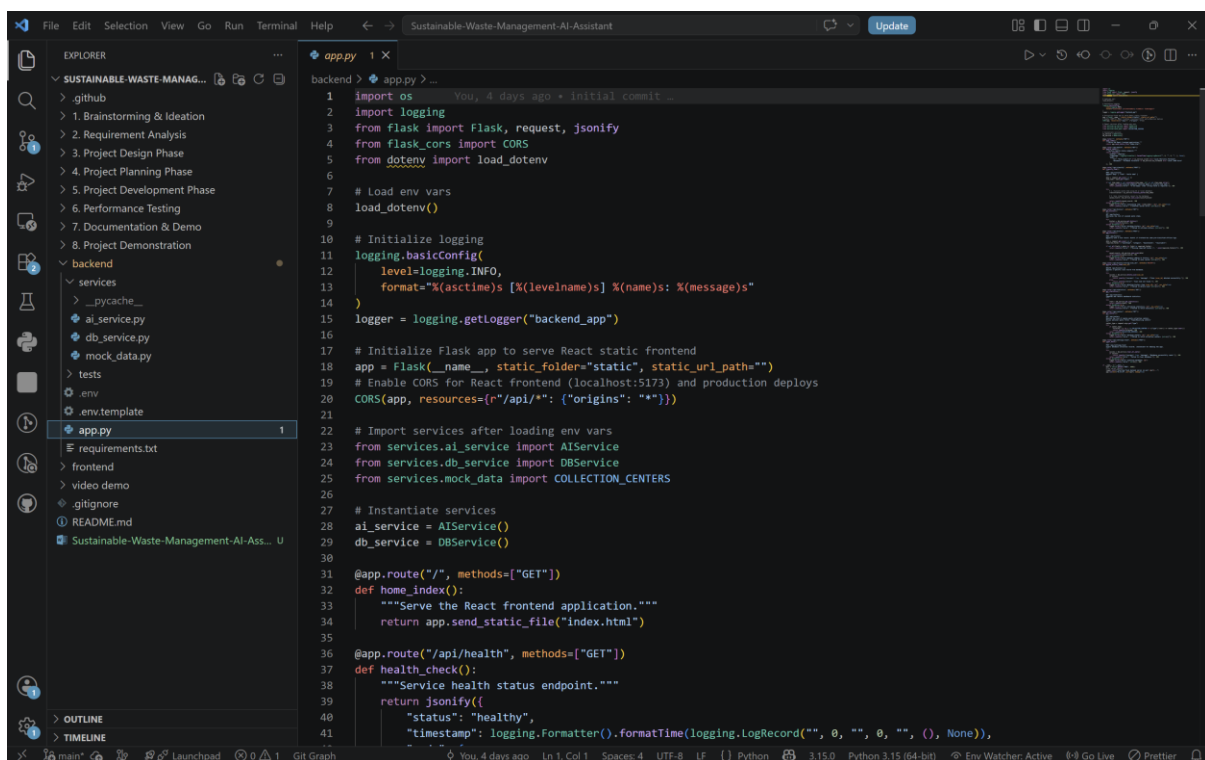
Milestone 3: Backend and Application Development

The Backend and Application Development phase focuses on implementing the server-side logic, Artificial Intelligence integration, cloud database connectivity, and application workflows of the EcoSense platform. During this phase, the backend was developed using Python Flask to process waste classification requests, communicate with the Groq AI service, manage Firebase and local JSON storage, and provide REST API endpoints for the React frontend. This architecture ensures secure communication between all application components while delivering fast and reliable responses.

Activity 3.1: Writing the Main Application Logic

The main application logic coordinates communication between the React frontend, Flask backend, Groq AI engine, Firebase database, and mapping services. When a user enters a waste item, the backend validates the input, sends a prompt to the Groq LLaMA 3.3-70B Versatile model, processes the AI response, stores the scan history, updates dashboard analytics, and returns detailed disposal guidance to the frontend. The backend also supports offline fallback mode using a local JSON database when cloud services are unavailable, ensuring uninterrupted functionality.

Figure 9.1: Main Backend Application Logic (Flask Backend)



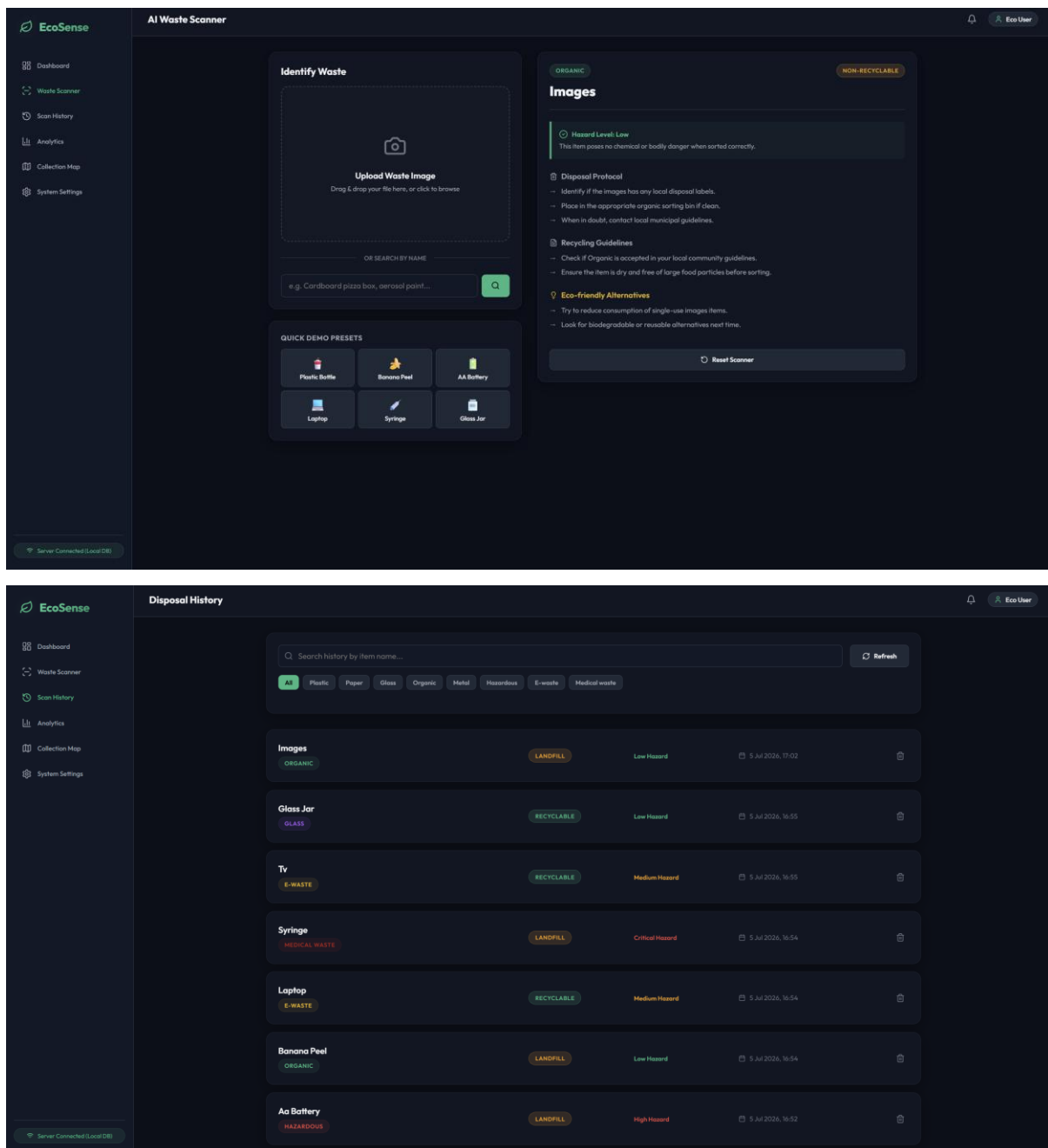
```

1 import os
2 import logging
3 from flask import Flask, request, jsonify
4 from flask_cors import CORS
5 from dotenv import load_dotenv
6
7 # Load env vars
8 load_dotenv()
9
10 # Initialize logging
11 logging.basicConfig(
12     level=logging.INFO,
13     format="%(asctime)s [%(levelname)s] %(name)s: %(message)s"
14 )
15 logger = logging.getLogger("backend_app")
16
17 # Initialize Flask app to serve React static frontend
18 app = Flask(__name__, static_folder="static", static_url_path="")
19 # Enable CORS for React frontend (localhost:5173) and production deploys
20 CORS(app, resources={r"/api/*": {"origins": "*"}})
21
22 # Import services after loading env vars
23 from services.ai_service import AIService
24 from services.db_service import DBService
25 from services.mock_data import COLLECTION_CENTERS
26
27 # Instantiate services
28 ai_service = AIService()
29 db_service = DBService()
30
31 @app.route("/", methods=["GET"])
32 def home_index():
33     """Serve the React frontend application."""
34     return app.send_static_file("index.html")
35
36 @app.route("/api/health", methods=["GET"])
37 def health_check():
38     """Service health status endpoint."""
39     return jsonify({
40         "status": "healthy",
41         "timestamp": logging.Formatter().formatTime(logging.LogRecord("", 0, "", 0, "", (), None)),
42     })
  
```

AI Waste Classification Integration

EcoSense integrates the **Groq API** with the **LLaMA 3.3-70B Versatile** language model to provide intelligent waste classification and disposal guidance. The backend sends the waste item entered by the user to the AI model, which analyzes the item and returns its waste category, recyclability status, disposal instructions, recycling recommendations, hazard warnings, and eco-friendly suggestions. This integration enables accurate, fast, and intelligent waste management assistance for everyday household items.

Figure 9.2: AI Waste Classification Output



The figure displays two screenshots of the EcoSense application interface, which is titled "AI Waste Scanner" and "Disposal History".

Top Screenshot: AI Waste Scanner - Identify Waste

- Left Sidebar:** Contains navigation links for Dashboard, Waste Scanner, Scan History, Analytics, Collection Map, and System Settings.
- Main Content Area:**
 - Identify Waste Section:** Features an "Upload Waste Image" button with a camera icon and a text input field for searching by name (e.g., "Cardboard pizza box, aerosol paint..."). Below this are "QUICK DEMO PRESETS" for Plastic Bottle, Banana Peel, AA Battery, Laptop, Syringe, and Glass Jar.
 - Results Panel:**
 - ORGANIC / NON-RECYCLABLE:** Shows "Hazard Level: Low" with a note: "This item poses no chemical or bodily danger when sorted correctly."
 - Disposal Protocol:**
 - Identify if the images has any local disposal labels.
 - Place in the appropriate organic sorting bin if clean.
 - When in doubt, contact local municipal guidelines.
 - Recycling Guidelines:**
 - Check if Organic is accepted in your local community guidelines.
 - Ensure the item is dry and free of large food particles before sorting.
 - Eco-friendly Alternatives:**
 - Try to reduce consumption of single-use images items.
 - Look for biodegradable or reusable alternatives next time.
 - Reset Scanner** button.

Bottom Screenshot: Disposal History

- Left Sidebar:** Same as the top screenshot.
- Main Content Area:**
 - Search:** "Search history by item name..." with a "Refresh" button.
 - Filters:** Tabs for All, Plastic, Paper, Glass, Organic, Metal, Hazardous, E-waste, and Medical waste.
 - History List:**

Item Name	Category	Disposal Status	Hazard Level	Date/Time	Action
Images	ORGANIC	LANDFILL	Low Hazard	5 Jul 2024, 17:02	🗑️
Glass Jar	GLASS	RECYCLABLE	Low Hazard	5 Jul 2024, 16:55	🗑️
Tv	E-WASTE	RECYCLABLE	Medium Hazard	5 Jul 2024, 16:55	🗑️
Syringe	MEDICAL WASTE	LANDFILL	Critical Hazard	5 Jul 2024, 16:54	🗑️
Laptop	E-WASTE	RECYCLABLE	Medium Hazard	5 Jul 2024, 16:54	🗑️
Banana Peel	ORGANIC	LANDFILL	Low Hazard	5 Jul 2024, 16:54	🗑️
Aa Battery	HAZARDOUS	LANDFILL	High Hazard	5 Jul 2024, 16:52	🗑️

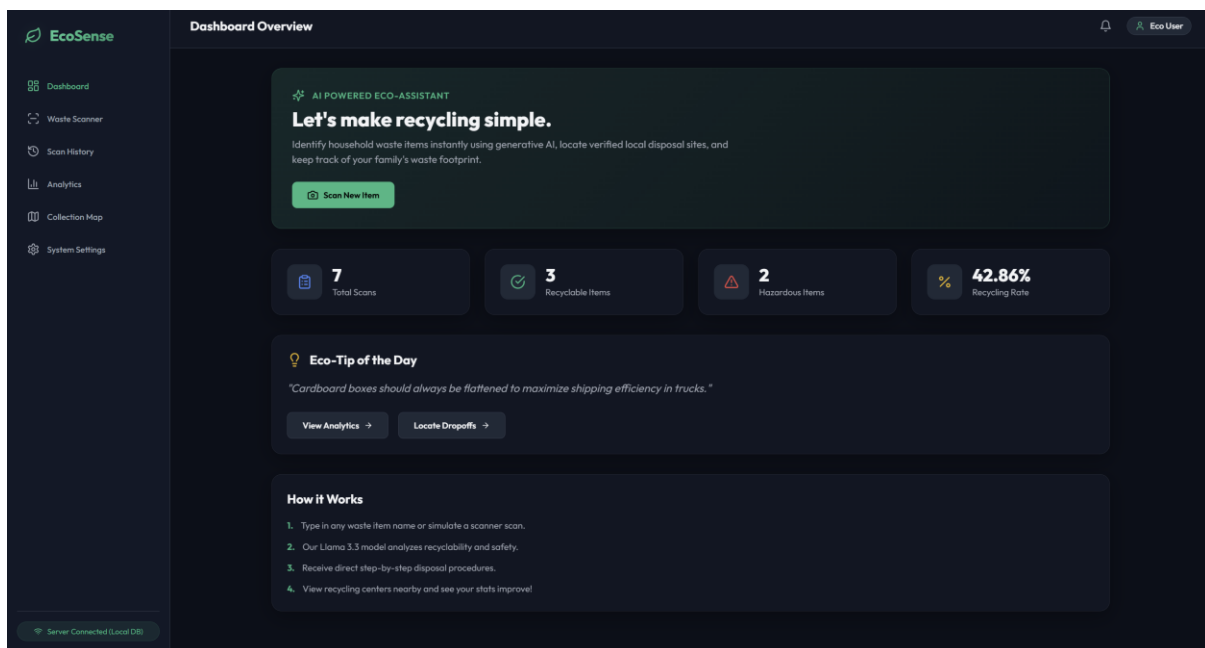
Milestone 4: Frontend Development

The Frontend Development phase focuses on designing and implementing a responsive, modern, and user-friendly interface for the EcoSense platform. The frontend was developed using **React.js**, **HTML5**, **CSS3**, and **JavaScript** to provide an interactive dashboard where users can scan waste items, view disposal instructions, analyze recycling statistics, locate nearby collection centers, and manage scan history. The interface is optimized for desktop, tablet, and mobile devices to ensure a seamless user experience.

Activity 4.1: Designing and Developing the User Interface

The EcoSense user interface was designed with simplicity, accessibility, and responsiveness in mind. A clean dashboard layout, navigation sidebar, responsive cards, interactive buttons, and real-time notifications were implemented to improve usability. The application allows users to quickly access all major modules including Waste Scanner, Analytics Dashboard, Collection Map, Scan History, and System Settings from a single interface.

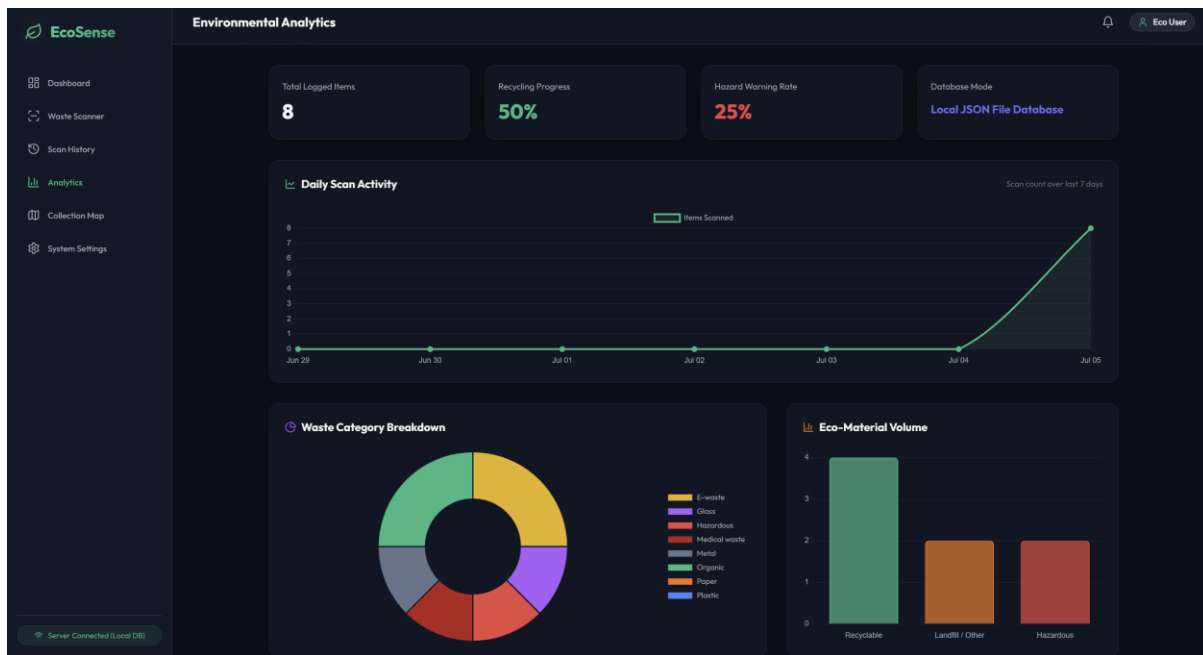
Figure 10.1: EcoSense Dashboard Interface



Dashboard Overview

The EcoSense dashboard provides users with a centralized overview of their waste management activities. It displays total scans, recyclable items, hazardous waste count, recycling rate, recent activity, and eco-friendly tips. Interactive cards and graphical summaries help users monitor their environmental impact while encouraging responsible waste disposal practices.

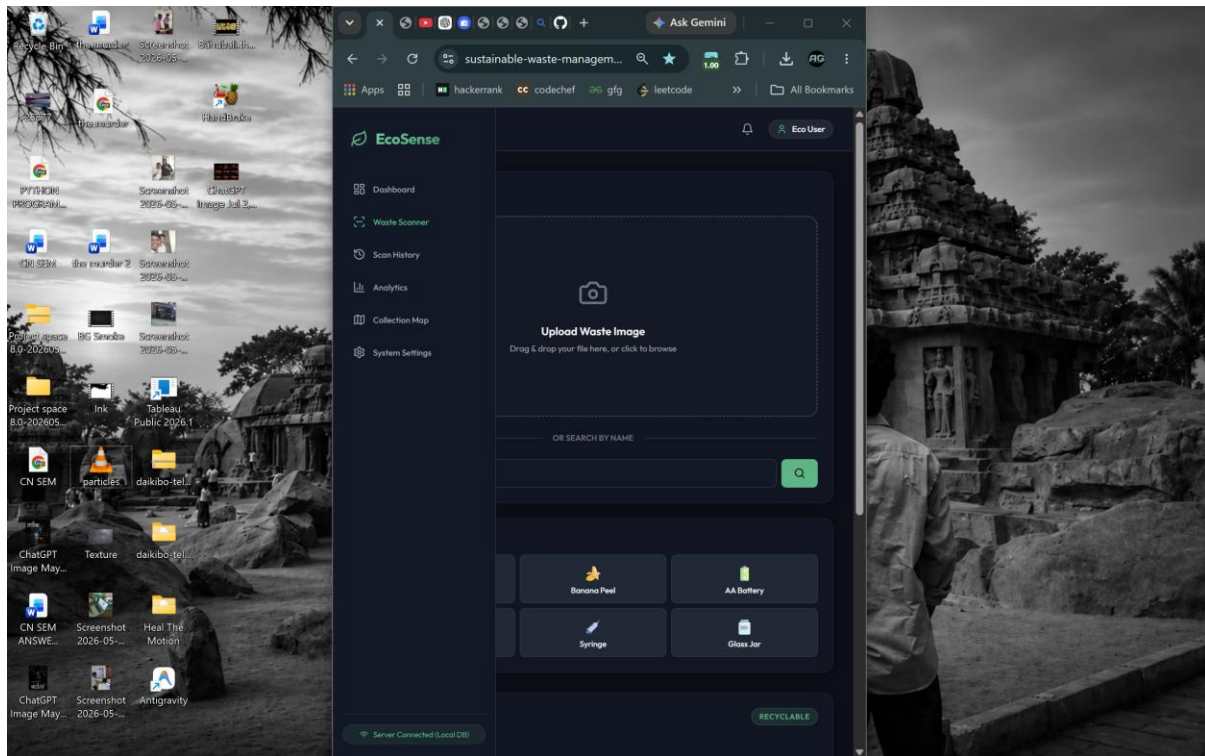
Figure 10.2: Dashboard Overview



Responsive Design

Responsive web design techniques were implemented using flexible layouts, CSS media queries, and adaptive React components. The interface automatically adjusts to different screen sizes, ensuring consistent functionality across desktops, laptops, tablets, and smartphones. This responsive design enhances accessibility and provides a smooth user experience on multiple devices.

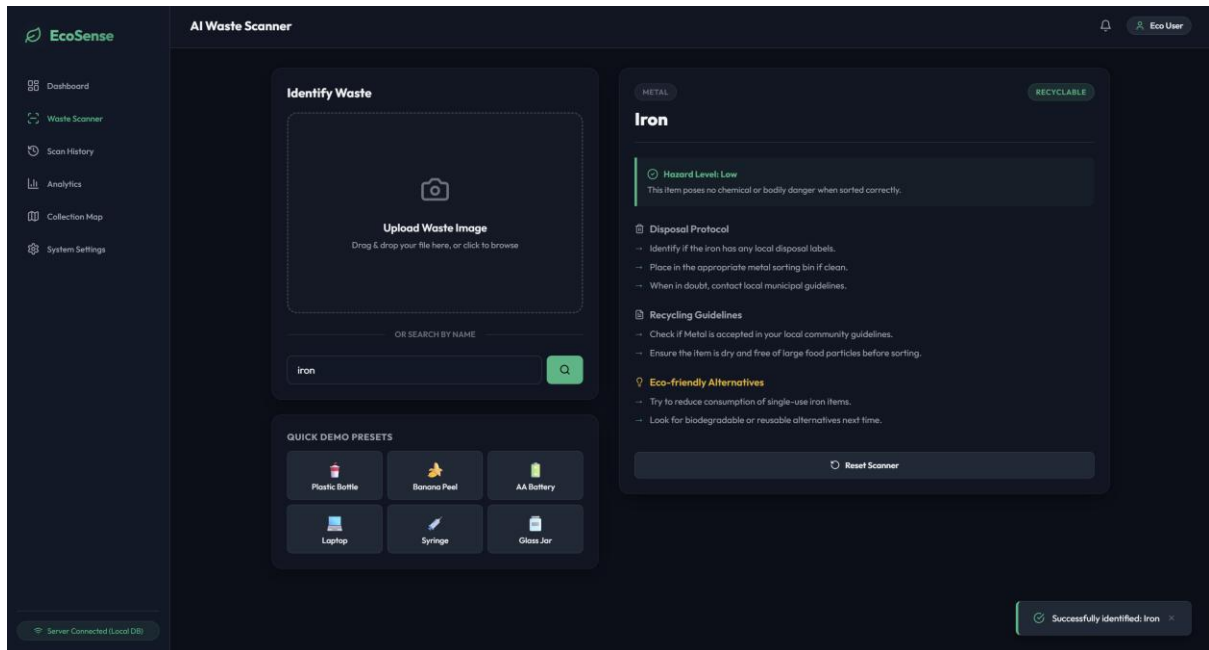
Figure 10.3: Responsive User Interface



Activity 4.2: Creating Dynamic Content Rendering with React

React.js was used to dynamically render waste classification results, dashboard statistics, scan history, analytics charts, and collection center information without requiring page reloads. State management and component-based architecture enable real-time updates, smooth navigation, and efficient rendering of application data. Interactive features such as live search, notifications, and dashboard updates significantly improve the overall user experience.

Figure 10.4: Dynamic Content Rendering



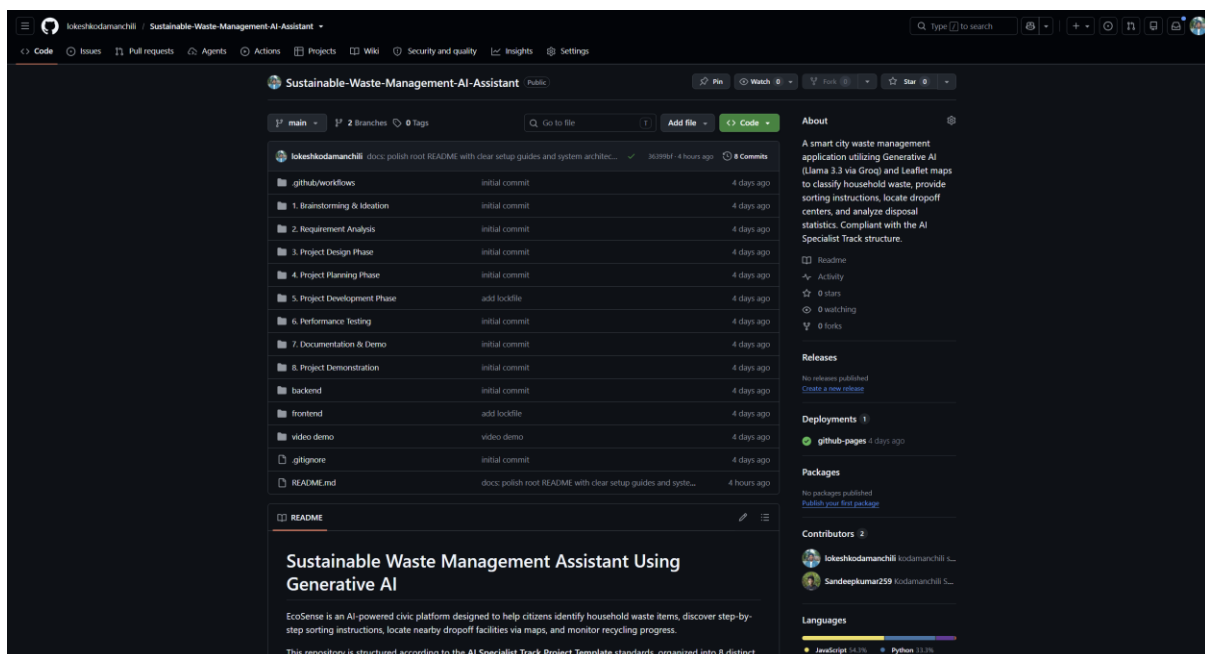
Milestone 5: Integration and Deployment

The Integration and Deployment phase combines the frontend, backend, Artificial Intelligence services, cloud database, and external APIs into a fully functional EcoSense platform. During this phase, all modules were integrated, tested, and deployed to ensure seamless communication between the React frontend, Flask backend, Groq AI service, Firebase database, and mapping services. Comprehensive testing was conducted to validate application performance, reliability, and user experience.

Activity 5.1: Preparing the Application for Local Deployment

Before deployment, the application environment was configured by organizing the frontend and backend project structure, installing required dependencies, configuring environment variables, and establishing connections with the Groq API and Firebase database. Local testing ensured that all services communicated correctly before the application was deployed to cloud platforms.

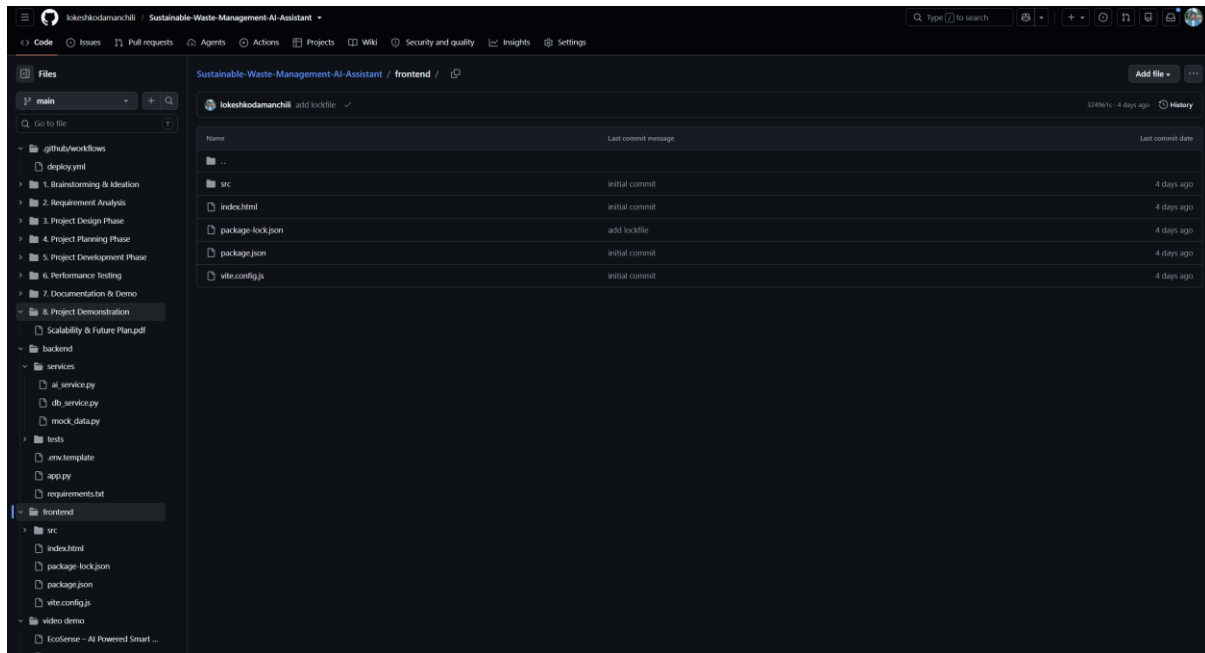
Figure 11.1: EcoSense Project Structure



Activity 5.2: Deploy the Frontend and Backend Services

The EcoSense frontend was deployed using **GitHub Pages**, while the Flask backend was deployed on **Render**. This deployment allows users to access the application through a web browser while enabling secure communication between the frontend, backend, Artificial Intelligence engine, Firebase database, and mapping services. The deployed application provides real-time waste classification, disposal guidance, analytics, and location services over the internet.

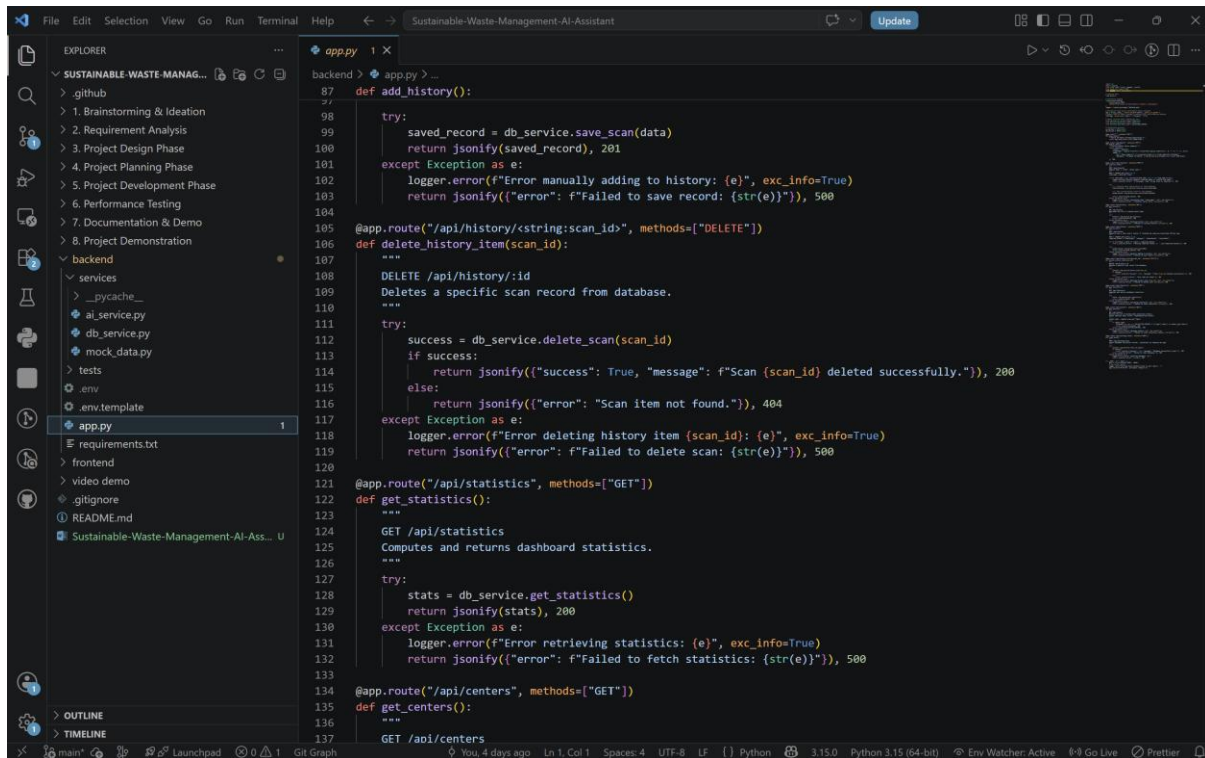
Figure 11.2: GitHub Repository and Deployment



Backend Service Deployment

The Flask backend processes user requests, communicates with the Groq AI API, stores waste scan history, and retrieves analytics data from Firebase or the local JSON database. REST API endpoints were configured to securely connect the React frontend with backend services. The backend also supports offline fallback mode to ensure uninterrupted application functionality even when cloud services are unavailable.

Figure 11.3: Flask Backend Deployment



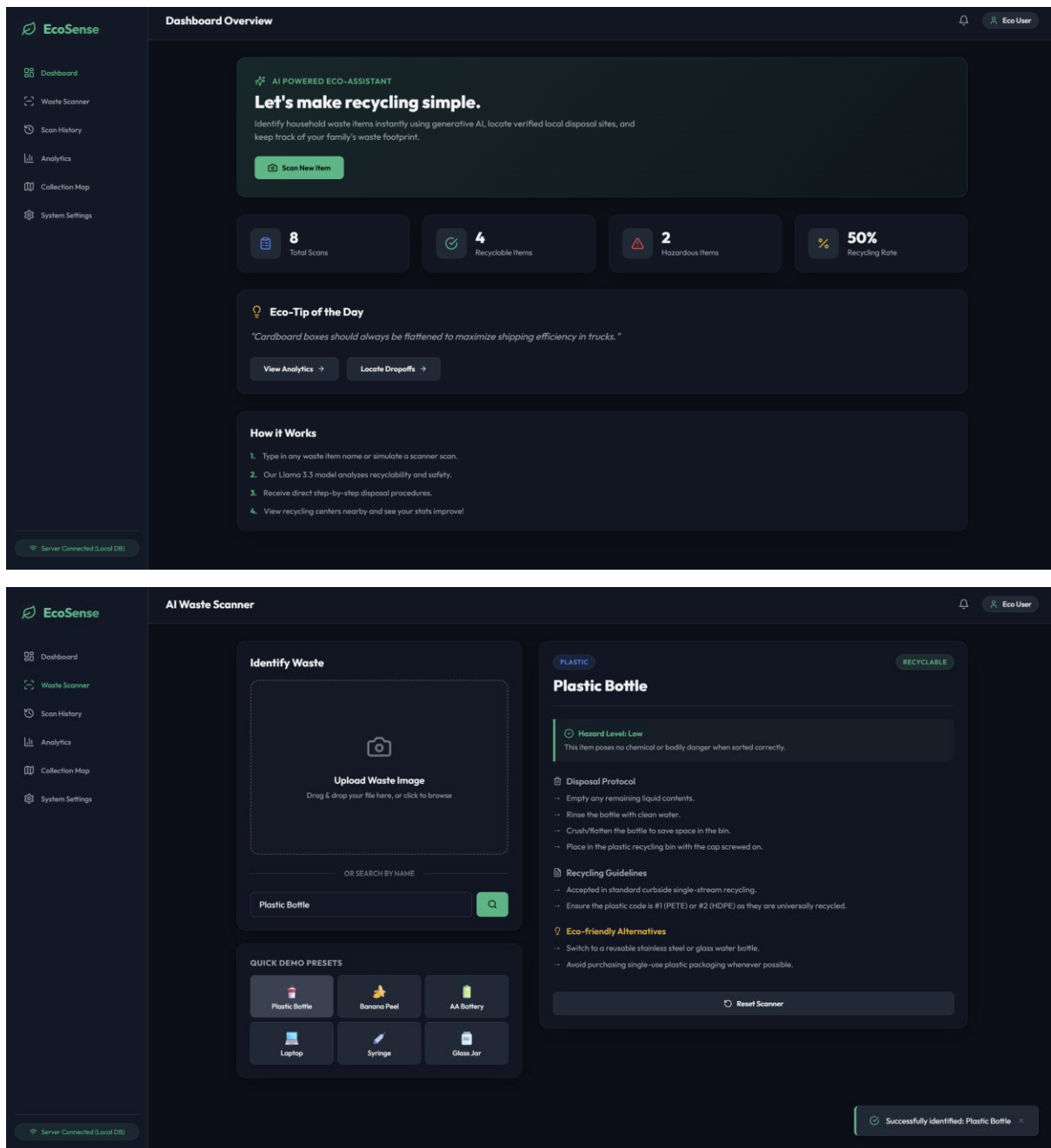
```

87 def add_history():
88     try:
89         saved_record = db_service.save_scan(data)
90         return jsonify(saved_record), 201
91     except Exception as e:
92         logger.error(f"Error manually adding to history: {e}", exc_info=True)
93         return jsonify({"error": f"Failed to save record: {str(e)}"}), 500
94
95 @app.route("/api/history/<string:scan_id>", methods=["DELETE"])
96 def delete_history_item(scan_id):
97     """
98     DELETE /api/history/:id
99     Deletes a specific scan record from database.
100     """
101     try:
102         success = db_service.delete_scan(scan_id)
103         if success:
104             return jsonify({"success": True, "message": f"Scan {scan_id} deleted successfully."}), 200
105         else:
106             return jsonify({"error": "Scan item not found."}), 404
107     except Exception as e:
108         logger.error(f"Error deleting history item {scan_id}: {e}", exc_info=True)
109         return jsonify({"error": f"Failed to delete scan: {str(e)}"}), 500
110
111 @app.route("/api/statistics", methods=["GET"])
112 def get_statistics():
113     """
114     GET /api/statistics
115     Computes and returns dashboard statistics.
116     """
117     try:
118         stats = db_service.get_statistics()
119         return jsonify(stats), 200
120     except Exception as e:
121         logger.error(f"Error retrieving statistics: {e}", exc_info=True)
122         return jsonify({"error": f"Failed to fetch statistics: {str(e)}"}), 500
123
124 @app.route("/api/centers", methods=["GET"])
125 def get_centers():
126     """
127     GET /api/centers
  
```

Activity 5.3: Validate Deployment and Test Live Workflow

Comprehensive testing was performed to validate waste classification, AI-generated disposal guidance, dashboard analytics, scan history management, collection center mapping, and overall application performance. The deployed application successfully processed multiple waste items, generated accurate AI responses, updated dashboard statistics, and displayed collection centers correctly. Functional testing confirmed reliable communication between all integrated services and ensured the system was ready for real-world usage.

Figure 11.4: Live Application Validation





Milestone 6: Conclusion

The Conclusion phase summarizes the successful development and implementation of the **EcoSense – Sustainable Waste Management Assistant Using Generative AI** platform. The application integrates modern web technologies, Artificial Intelligence, cloud services, interactive mapping, and analytics into a single smart waste management solution. EcoSense provides users with an intelligent, user-friendly, and scalable platform that promotes responsible waste disposal, recycling awareness, and environmental sustainability through AI-powered assistance.

Activity 6.1: Summarize the Overall Project Objectives and Achievements

EcoSense successfully achieves its primary objective of simplifying household waste management through Artificial Intelligence and modern web technologies. The application combines AI-powered waste classification, recycling guidance, interactive collection center mapping, analytics dashboard, and scan history management into one centralized platform. Throughout the development process, the project demonstrated successful integration of React.js, Python Flask, Groq AI, Firebase-compatible architecture, Leaflet.js, and Chart.js while maintaining an intuitive user interface and reliable system performance.

Activity 6.2: Evaluate the Performance and Reliability

The application was tested across all major functional modules, including AI waste classification, disposal guidance generation, dashboard analytics, scan history management, interactive maps, and backend API communication. Functional testing confirmed accurate AI responses, responsive user interaction, reliable backend processing, and efficient data management. The modular architecture also improves maintainability, scalability, and future enhancements.

Activity 6.3: Highlight Project Benefits

EcoSense provides several practical benefits, including intelligent waste identification, environmentally responsible disposal guidance, increased recycling awareness, centralized waste management records, cloud-enabled accessibility, and interactive analytics. The application reduces manual effort, improves public awareness of proper waste segregation, and demonstrates how Artificial Intelligence can contribute to smart city sustainability and environmental protection.

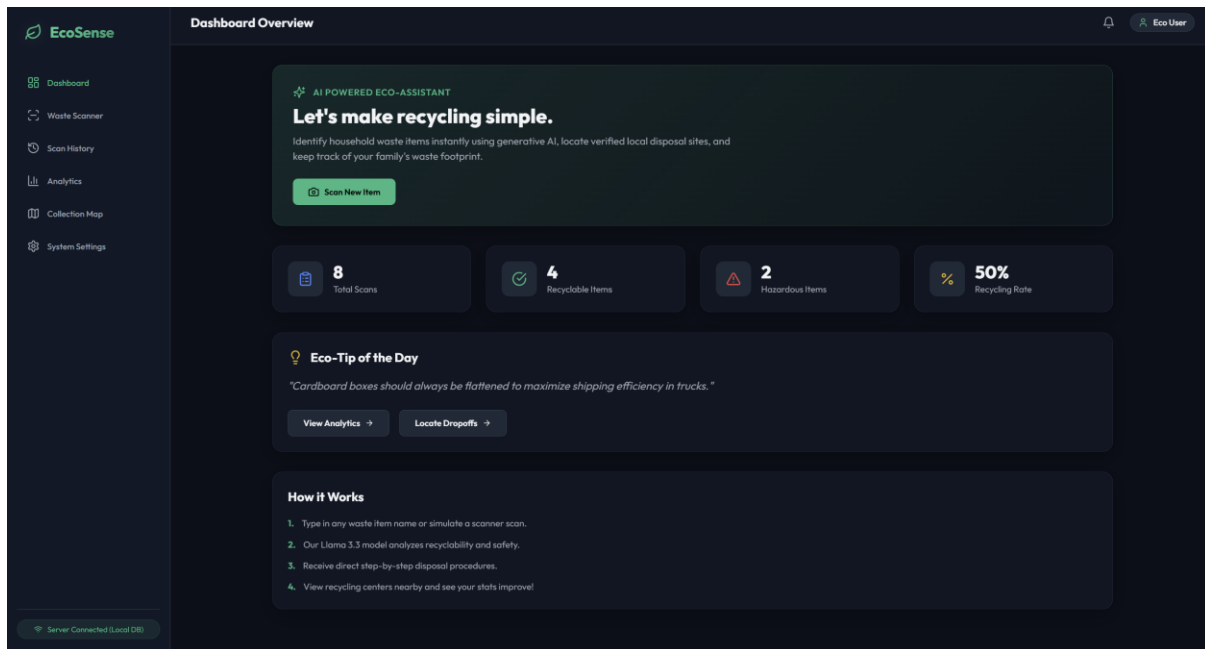


Figure 12.1: Final EcoSense Application

Activity 6.4: Project Limitations and Future Enhancements

The current implementation focuses on AI-powered waste classification and smart recycling assistance for household waste items. Future enhancements may include image-based waste recognition using computer vision, IoT-enabled smart dustbin integration, mobile application support for Android and iOS, multilingual assistance, voice-based interaction, QR-code enabled recycling systems, carbon footprint estimation, and predictive waste analytics to further improve sustainable waste management practices.

Activity 6.5: Final Documentation, Testing, and Review

The EcoSense project was successfully completed following a structured software development lifecycle, including requirement analysis, system design, implementation, testing, deployment, and documentation. All functional modules were verified, screenshots were documented, and the complete project workflow was reviewed to ensure accuracy, consistency, and readiness for project evaluation. The project demonstrates the practical application of Artificial Intelligence in solving real-world environmental challenges while promoting sustainable waste management practices.

Final Conclusion

EcoSense – Sustainable Waste Management Assistant Using Generative AI demonstrates how Artificial Intelligence can support smart waste management and environmental sustainability. By integrating React.js, Python Flask, Groq AI, interactive maps, and analytics, the application provides an efficient platform for waste classification, recycling guidance, and responsible waste disposal. The project promotes eco-friendly practices and highlights the role of AI in building smarter and greener cities.